

PULS COFFEE

Mobile Ordering App

Complete Developer Specification — Version 3.0



SwiftUI iOS • NestJS + TypeScript • PostgreSQL • AWS
Clover POS • Stripe • AI Layer (Phase 2) • Telegram Alerts
Staff Dashboard • 50 Features • 15 Golden Rules

Table of Contents

PART 1 Project Overview — Puls Coffee

- 1.1 What We're Building & Why
- 1.2 North Star Metrics
- 1.3 Two-Sided Platform

PART 2 Final Architecture Decision

- 2.1 Architecture Overview
- 2.2 Final Tech Stack
- 2.3 What We Decided NOT to Build (and Why)

PART 3 Database — All Tables

- 3.1 Location & User Tables
- 3.2 Menu & Inventory Tables
- 3.3 Order, Payment & Refund Tables
- 3.4 AI & Behaviour Tables (Phase 2)
- 3.5 Loyalty, Outbox & Feature Flags

PART 4 Backend — Modules, Workers & API

- 4.1 NestJS Module & File Structure
- 4.2 Workers & Queue Architecture
- 4.3 REST API Endpoints — Complete List
- 4.4 Role-Based Access Control
- 4.5 Rate Limiting

PART 5 Core Flows — Exact Implementation

- 5.1 Three State Machines
- 5.2 Checkout & Stripe Flow (Step by Step)
- 5.3 Outbox Pattern
- 5.4 Clover Sync & Dead Letter Queue
- 5.5 Pickup Time & Location Hours Logic

PART 6 iOS App — SwiftUI

- 6.1 Project Structure
- 6.2 Key Rules
- 6.3 Performance, Caching & Offline

PART 7 Feature Roadmap — Prioritised

- 7.1 Phase 1 — Core Ordering (Weeks 1-8)
- 7.2 Phase 2 — Retention & Intelligence
- 7.3 Phase 3 — Delight & Premium UX
- 7.4 Phase 4 — Scale & SaaS
- 7.5 Features Removed or Delayed (with reasons)

PART 8 AI Layer — Full Specification

- 8.1 When to Build AI (not before this)
- 8.2 AI Architecture & Memory System
- 8.3 Customer Segments & Marketing Engine

8.4 Pricing Psychology

8.5 Engagement Mechanics

PART 9 Staff Dashboard & Telegram Bot

PART 10 Infrastructure, Security & Monitoring

PART 11 8-Week Build Plan

PART 12 Cost Estimates

PART 13 15 Golden Rules — Non-Negotiable

PART 1 — PROJECT OVERVIEW

1.1 What We're Building & Why

Puls Coffee is an independent coffee shop that needs a mobile ordering iOS app. The goal is not to build a 'futuristic AI app'. The goal is to build a reliable ordering system that customers trust every morning — then layer intelligence on top once real data exists.

Core philosophy: Build the smallest reliable system possible. Every feature must improve ordering, improve operations, or improve retention — or it does not exist yet.

Priority	Goal	Why It Matters
#1	Menu loads instantly	First impression. Cached locally. Never wait for network.
#2	Checkout never duplicates	Idempotency keys + Stripe webhook truth. Money on the line.
#3	Order status always accurate	Three separate enums. iOS only displays, never decides.
#4	Clover sync is reliable	Retry (0s→30s→2m→10m) + DLQ + Telegram. Staff not surprised.
#5	Staff have full control	Pause orders, mark sold-out, set wait time, issue refunds.
#6	Owner has visibility	Telegram alerts. No need for separate dashboard at launch.
#7	AI-ready from day one	DB schema prepared. Build AI in Phase 2 when data exists.
#8	Multi-location from day one	Every record scoped to location_id. Never rewrite for store #2.

1.2 North Star Metrics (what we measure, not feature count)

ChatGPT Pro's most important advice: replace the '50 features' mindset with 6 numbers that matter.

Metric	Target at Launch	Why This One
Daily Active Users	Growing week-over-week	People using it = it's working
Repeat Weekly Customers	60%+ return within 7 days	Habit formation = retention
Average Order Frequency	3+ orders/week per active user	Core business metric
Checkout Success Rate	99%+	Zero tolerance for payment failures
Subscription Retention (Phase 2)	80%+ monthly	Guaranteed revenue foundation
Attach Rate (drink + food)	25%+ of orders	Bundle intelligence ROI signal

1.3 Two-Sided Platform

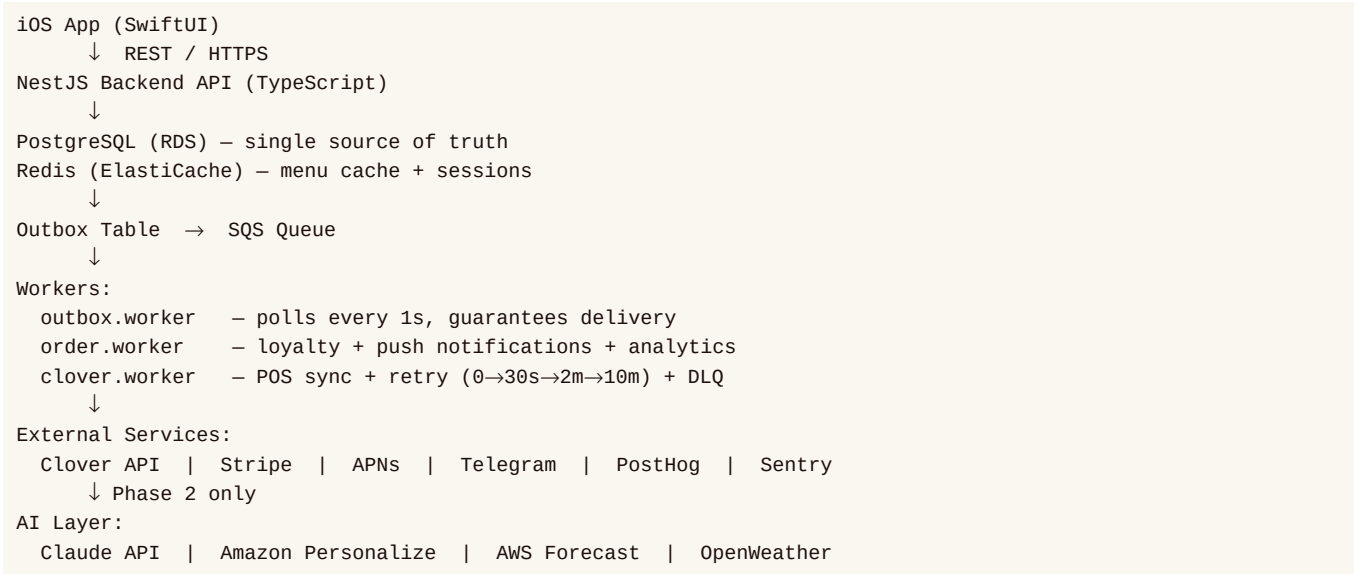
Side	Users	What They Get	Built When
Customer iOS App	Coffee buyers	Menu, order, pay, track, loyalty, AI (Phase 2)	Week 6
Staff Web Dashboard	Baristas & managers	Live queue, accept/ready/cancel, inventory, refunds	Week 7
Owner Telegram Bot	Shop owner	New order alerts, failures, daily revenue summary	Week 5

Clover POS	Baristas (unchanged)	Every mobile order arrives automatically	Week 4
------------	-------------------------	--	--------

PART 2 — FINAL ARCHITECTURE DECISION

2.1 Architecture Overview

Simple. No microservices. No EventBridge complexity. No 20 Lambdas. No GraphQL. One API. One worker. One database. Proven pattern.



2.2 Final Tech Stack

Layer	Technology	Notes
iOS App	SwiftUI (iOS 16+)	Native. Cart in memory. Menu cached locally.
Backend API	NestJS + TypeScript	One service. Easier to debug than 20 Lambdas.
Database	PostgreSQL 15 (AWS RDS)	ACID transactions. Complex order relationships. Easy reporting.
Cache	Redis (ElastiCache)	Menu cache (10min TTL). Idempotency keys (24h TTL).
Queue	AWS SQS	6 critical events only. DLQ after 4 failed attempts.
Payments	Stripe	PaymentIntents + webhooks. Webhook = only source of truth.
POS	Clover REST API	Menu import + order sync. Keys in Parameter Store.
Auth	JWT + AWS Cognito	Separate pools: customers vs staff.
Push	Apple APNs	ORDER_READY, ORDER_CANCELLED, promos.
Alerts	Telegram Bot API	Owner: new orders, failures, daily summary.
Errors	Sentry	iOS + backend. Day one. Non-negotiable.
Analytics	PostHog	Behaviour funnels. Free up to 1M events/mo.
Secrets	AWS Parameter Store	Clover + Stripe keys encrypted. Never in .env files.
Hosting	AWS ECS Fargate	Containers. No server management.
CDN	S3 + CloudFront	Menu images. Cache-Control: 1 year.

Staff UI	React + TypeScript	Hosted on AWS Amplify (free tier).
----------	--------------------	------------------------------------

2.3 What We Decided NOT to Build (and Why)

This section is as important as what we do build. Based on three rounds of expert review.

Removed/Delayed	Reason
AppSync / GraphQL	Adds complexity with no MVP benefit. REST is sufficient and simpler to debug.
EventBridge (for tiny events)	Noise and cost. Use only for 6 critical business events. PostHog handles the rest.
20 separate Lambdas	Hard to debug. One NestJS service + 3 workers is cleaner and faster to build.
Server-side cart	Cart lives on iOS until checkout. No abandoned cart needed in Phase 1.
Mood detection	Becomes creepy quickly. 'Rough Monday?' feels invasive. Removed permanently.
Bluetooth beacons	Operational nightmare. Hardware dependency. Remove for years.
Apple Watch app	Tiny ROI. Low on every priority list from all reviewers.
Conversational AI	Most people know what they want. Low ROI in Phase 1.
Individualized surge pricing	Customers hate feeling manipulated. Only discounts off base price allowed.
AR menu	Phase 4 at earliest. Not a coffee shop priority.
AI offer engine (day one)	Start rule-based. 'Simple beats smart.' AI offers in Phase 2 only.
Infrastructure as Code (day one)	Use when deployment becomes serious. Not a blocker for launch.

PART 3 — DATABASE — ALL TABLES

All money = INTEGER CENTS (650 not 6.50). All IDs = UUID. Every mutable table has created_at / updated_at. Enums as PostgreSQL ENUM types. synchronize: true in development only.

3.1 Location & User Tables

locations	
id	UUID PRIMARY KEY
name	TEXT NOT NULL
address	TEXT NOT NULL
phone	TEXT
timezone	TEXT DEFAULT 'America/New_York'
active	BOOLEAN DEFAULT TRUE
created_at	TIMESTAMPTZ DEFAULT NOW()

location_hours	
id	UUID PK
location_id	UUID FK → locations
day_of_week	INT 0=Sunday, 6=Saturday
open_time	TIME NOT NULL
close_time	TIME NOT NULL
is_closed	BOOLEAN DEFAULT FALSE

location_settings	
location_id	UUID PK FK → locations
mobile_ordering_paused	BOOLEAN DEFAULT FALSE
current_wait_minutes	INT DEFAULT 5
scheduled_ordering	BOOLEAN DEFAULT TRUE
max_schedule_days	INT DEFAULT 7
updated_at	TIMESTAMPTZ

customers	
id	UUID PK
email	TEXT UNIQUE NOT NULL
phone	TEXT
full_name	TEXT NOT NULL
cognito_id	TEXT UNIQUE
password_hash	TEXT used if not Cognito
loyalty_points	INT DEFAULT 0
loyalty_tier	TEXT DEFAULT 'BRONZE'
stripe_customer_id	TEXT
push_token	TEXT APNs device token
last_visit_at	TIMESTAMPTZ
created_at	TIMESTAMPTZ

staff_users	
id	UUID PK
location_id	UUID FK → locations
email	TEXT UNIQUE NOT NULL
full_name	TEXT NOT NULL
role	staff_role_enum OWNER MANAGER BARISTA
password_hash	TEXT NOT NULL
cognito_id	TEXT UNIQUE
active	BOOLEAN DEFAULT TRUE
created_at	TIMESTAMPTZ

3.2 Menu & Inventory Tables

menu_categories	
id	UUID PK
location_id	UUID FK → locations
name	TEXT NOT NULL
sort_order	INT DEFAULT 0
active	BOOLEAN DEFAULT TRUE

menu_items	
id	UUID PK
category_id	UUID FK → menu_categories
clover_item_id	TEXT Clover source ID – used for sync
name	TEXT NOT NULL
description	TEXT
base_price_cents	INT NOT NULL INTEGER CENTS – never float
image_url	TEXT S3 + CloudFront URL
active	BOOLEAN DEFAULT TRUE
created_at	TIMESTAMPTZ
updated_at	TIMESTAMPTZ
modifier_groups	
id	UUID PK
item_id	UUID FK → menu_items
name	TEXT e.g. 'Milk', 'Size', 'Extras'
required	BOOLEAN DEFAULT FALSE
multi_select	BOOLEAN DEFAULT FALSE
sort_order	INT DEFAULT 0
modifiers	
id	UUID PK
group_id	UUID FK → modifier_groups
clover_mod_id	TEXT
name	TEXT e.g. 'Oat Milk', 'Large'
price_cents	INT DEFAULT 0 upcharge – INTEGER CENTS
sort_order	INT DEFAULT 0
active	BOOLEAN DEFAULT TRUE
inventory	
id	UUID PK
item_id	UUID FK → menu_items
location_id	UUID FK → locations
available	BOOLEAN DEFAULT TRUE
quantity_left	INT NULL = unlimited
sold_out_at	TIMESTAMPTZ
updated_by	UUID FK → staff_users
updated_at	TIMESTAMPTZ
UNIQUE	(item_id, location_id)

pricing_rules	
id	UUID PK
location_id	UUID FK → locations
tax_rate_bps	INT 875 = 8.75% basis points – avoids float
tip_options	INT[] DEFAULT '{15,18,20,25}'
active	BOOLEAN DEFAULT TRUE
updated_at	TIMESTAMPTZ

3.3 Order, Payment & Refund Tables

orders	
id	UUID PK
customer_id	UUID FK → customers
location_id	UUID FK → locations
idempotency_key	TEXT UNIQUE NOT NULL client: userId+cartHash+timestamp
order_status	order_status_enum see state machine in Part 5
payment_status	payment_status_enum INDEPENDENT of order status
clover_sync_status	clover_sync_status_enum INDEPENDENT of payment
pickup_type	TEXT ASAP SCHEDULED
scheduled_pickup_at	TIMESTAMPTZ
estimated_ready_at	TIMESTAMPTZ
subtotal_cents	INT NOT NULL
modifier_cents	INT NOT NULL DEFAULT 0
discount_cents	INT NOT NULL DEFAULT 0
tax_cents	INT NOT NULL
tip_cents	INT NOT NULL DEFAULT 0
total_cents	INT NOT NULL
stripe_payment_id	TEXT
clover_order_id	TEXT
notes	TEXT
created_at	TIMESTAMPTZ
updated_at	TIMESTAMPTZ

order_items	
id	UUID PK
order_id	UUID FK → orders
menu_item_id	UUID FK → menu_items
quantity	INT NOT NULL DEFAULT 1
unit_price_cents	INT NOT NULL price snapshot at time of order
modifiers	JSONB DEFAULT '[]' [{modifierId, name, priceCents}]

order_events	
id	UUID PK
order_id	UUID FK → orders
from_status	TEXT nullable
to_status	TEXT NOT NULL
reason	TEXT
created_by	TEXT 'system' 'customer' staff_id
metadata	JSONB
created_at	TIMESTAMPTZ DEFAULT NOW()
payments	
id	UUID PK
order_id	UUID FK → orders
stripe_payment_id	TEXT UNIQUE NOT NULL
amount_cents	INT NOT NULL
payment_status	payment_status_enum
stripe_response	JSONB full Stripe payload for debugging
created_at	TIMESTAMPTZ
updated_at	TIMESTAMPTZ
refunds	
id	UUID PK
order_id	UUID FK → orders
payment_id	UUID FK → payments
stripe_refund_id	TEXT UNIQUE NOT NULL
amount_cents	INT NOT NULL supports partial refunds
reason	TEXT NOT NULL
created_by	UUID FK → staff_users
created_at	TIMESTAMPTZ
clover_sync_log	
id	UUID PK
order_id	UUID FK → orders
attempt_number	INT NOT NULL 1..4
sync_status	clover_sync_status_enum
request_payload	JSONB
response_payload	JSONB
error_message	TEXT
attempted_at	TIMESTAMPTZ

3.4 AI & Behaviour Tables (created now, populated Phase 2)

These tables exist from day one because historical data is invaluable later. No logic populates them until Phase 2.

customer_ai_profiles	
customer_id	UUID PK FK → customers
preferred_milk	TEXT learned from order history
preferred_size	TEXT
preferred_temp	TEXT hot iced both
disliked_items	TEXT[] never suggest these
dietary_flags	TEXT[] gluten-free, dairy-free, vegan
price_segment	TEXT budget mid premium
impulse_buyer	BOOLEAN
deal_responsive	BOOLEAN
adventurousness	DECIMAL 0.0-1.0
churn_score	DECIMAL 0.0-1.0 – higher = at risk
lifetime_value_cents	INT
avg_order_value_cents	INT
usual_order_time	TIME
usual_order_days	INT[]
notification_open_rate	DECIMAL
recommendation_accept_rate	DECIMAL
last_calculated_at	TIMESTAMPTZ
behavior_events	
id	UUID PK
customer_id	UUID FK → customers
event_type	TEXT menu_viewed item_viewed cart_added cart_removed offer_seen offer_redeemed app_opened
item_id	UUID nullable
duration_seconds	INT
metadata	JSONB
created_at	TIMESTAMPTZ

ai_decisions	
id	UUID PK
customer_id	UUID FK → customers
trigger	TEXT morning_routine churn_risk weather flash_deal
decision_type	TEXT recommendation offer notification
recommendation_item_id	UUID nullable
offer_type	TEXT free_drink discount points bundle
offer_value_cents	INT
confidence	DECIMAL 0.0-1.0
was_shown	BOOLEAN DEFAULT FALSE
was_accepted	BOOLEAN
revenue_generated_cents	INT tracked after the fact
created_at	TIMESTAMPTZ
resolved_at	TIMESTAMPTZ

offers	
id	UUID PK
customer_id	UUID FK → customers
type	TEXT winback loyalty_reward bundle flash birthday surprise
value_cents	INT cost to business
description	TEXT
sent_at	TIMESTAMPTZ
opened_at	TIMESTAMPTZ
redeemed_at	TIMESTAMPTZ
expires_at	TIMESTAMPTZ
revenue_attributed_cents	INT revenue in 7 days after offer

3.5 Loyalty, Outbox & Feature Flags

loyalty_accounts	
customer_id	UUID PK FK → customers
points_balance	INT DEFAULT 0
lifetime_points	INT DEFAULT 0
tier	TEXT BRONZE SILVER GOLD PLATINUM
updated_at	TIMESTAMPTZ

loyalty_transactions	
id	UUID PK
customer_id	UUID FK → customers
order_id	UUID FK → orders
points_delta	INT positive=earn, negative=redeem
reason	TEXT NOT NULL
created_at	TIMESTAMPTZ

outbox_events	
id	UUID PK
event_type	TEXT ORDER_PAID ORDER_CANCELLED ORDER_READY ORDER_PICKED_UP REFUND_CREATED ITEM_OUT_OF_STOCK
payload	JSONB NOT NULL
status	TEXT PENDING PROCESSED DEAD
attempts	INT DEFAULT 0
last_error	TEXT
processed_at	TIMESTAMPTZ
created_at	TIMESTAMPTZ

feature_flags	
key	TEXT PK
enabled	BOOLEAN DEFAULT FALSE
rollout_pct	INT DEFAULT 0 0-100% of users
description	TEXT
updated_at	TIMESTAMPTZ

Flag Key	Default	Description
loyalty	TRUE	Loyalty points and tier system
scheduled_ordering	TRUE	Allow future pickup scheduling
ai_recommendations	FALSE	AI-powered home screen suggestions
dynamic_pricing	FALSE	Time and demand based pricing
subscriptions	FALSE	Monthly \$29/mo coffee pass
group_ordering	FALSE	Shared cart / group orders
conversational_ai	FALSE	Chat-based order interface
predictive_preorder	FALSE	Geofence-triggered pre-ordering
apple_watch_app	FALSE	Apple Watch companion app
flash_deals	FALSE	AI-triggered time-limited offers
bundle_suggestions	FALSE	AI personalised bundles
web_ordering	FALSE	Browser-based ordering

4.1 NestJS Module & File Structure

```
puls-coffee-backend/
src/
  database/
    entities.ts           All TypeORM entities + all enums
    data-source.ts        TypeORM DataSource config
    seeds/
    feature-flags.seed.ts
  modules/
    auth/                 jwt.strategy.ts | roles.guard.ts | auth.service.ts
    users/                users.controller.ts | users.service.ts
    locations/            locations.service.ts | hours.service.ts (open/closed)
    menu/                 menu.controller.ts | menu.service.ts | menu.cache.ts (Redis)
    pricing/              pricing.service.ts (ALL math in cents) | tax.service.ts
    inventory/            inventory.controller.ts | inventory.service.ts
    checkout/             checkout.controller.ts | checkout.service.ts | idempotency.guard.ts
    orders/               orders.controller.ts | orders.service.ts | state-machine.ts
    payments/             stripe.controller.ts | stripe.service.ts | webhook.guard.ts
    refunds/              refunds.controller.ts | refunds.service.ts
    clover/               clover.service.ts | clover-sync.service.ts | menu-import.service.ts
    loyalty/              loyalty.service.ts | tiers.service.ts
    notifications/        push.service.ts (APNs) | telegram.service.ts
    admin/                admin.controller.ts | admin.service.ts
    analytics/            posthog.service.ts
  workers/
    outbox.worker.ts      polls outbox_events every 1 second
    order.worker.ts       routes ORDER_* events → loyalty + push + analytics
    clover.worker.ts      POS sync + retry (0→30s→2m→10m) + DLQ + Telegram
  common/
    guards/               roles.guard.ts | rate-limit.guard.ts | idempotency.guard.ts
    filters/              exception.filter.ts (Sentry)
    pipes/                validation.pipe.ts
    decorators/           roles.decorator.ts
  app.module.ts
  main.ts                Sentry init FIRST – before any other import
  instrument.ts           Sentry instrumentation file
```

4.2 Workers & Event Architecture

Worker	How Triggered	What It Does
outbox.worker	Polls DB every 1 second	Finds PENDING outbox_events. Publishes each to SQS. Marks PROCESSED or DEAD (after 5 fails).
order.worker	SQS ORDER_PAID event	Triggers loyalty points calculation + push notification + PostHog analytics event.
clover.worker	SQS ORDER_PAID event	Creates order in Clover. Retries 4x (0s→30s→2m→10m). On final fail: DLQ + MANUAL_REQUIRED + Telegram.

Six SQS Events — Business Critical Only

Event	Triggers
ORDER_PAID	Clover sync push notification loyalty points Telegram alert PostHog

ORDER_CANCELLED	Refund if needed push notification Telegram analytics
ORDER_READY	Push notification: 'Your coffee is ready ■'
ORDER_PICKED_UP	Close order analytics update customer last_visit_at
REFUND_CREATED	Stripe refund update order status push notification Telegram
ITEM_OUT_OF_STOCK	Hide from menu instantly Telegram alert to owner

4.3 REST API Endpoints — Complete List

Method	Path	Auth	Description
GET	/api/v1/menu	Public	Full menu + categories + modifiers. Redis cached 10min.
GET	/api/v1/menu/items/:id	Public	Single item with all modifier groups.
POST	/api/v1/auth/register	Public	Customer registration. Returns JWT.
POST	/api/v1/auth/login	Public	Rate limited: 5/min per IP.
POST	/api/v1/auth/refresh	Public	Refresh JWT access token.
POST	/api/v1/checkout	Customer	Idempotency-Key header required. Validates items, calculates price, creates Stripe PaymentIntent.
GET	/api/v1/orders/:id	Customer	Order status + items + estimated ready time.
GET	/api/v1/orders/my	Customer	Order history. Paginated.
POST	/api/v1/orders/:id/cancel	Customer	DRAFT status only.
GET	/api/v1/loyalty/my	Customer	Points balance, tier, transaction history.
POST	/api/v1/stripe/webhook	Stripe sig	Payment source of truth. Verify Stripe-Signature header.
GET	/api/v1/admin/orders	BARISTA+	Live order queue for location.
POST	/api/v1/admin/orders/:id/accept	BARISTA+	PAID → ACCEPTED.
POST	/api/v1/admin/orders/:id/progress	BARISTA+	ACCEPTED → IN_PROGRESS.
POST	/api/v1/admin/orders/:id/ready	BARISTA+	→ READY. Push sent to customer.
POST	/api/v1/admin/orders/:id/cancel	MANAGER +	Cancel with reason.
POST	/api/v1/admin/orders/:id/refund	MANAGER +	Full or partial refund via Stripe.
POST	/api/v1/admin/items/:id/sold-out	BARISTA+	Toggle unavailable instantly.
POST	/api/v1/admin/items/:id/available	BARISTA+	Restore availability.
POST	/api/v1/admin/ordering/pause	MANAGER +	Pause all mobile orders.
POST	/api/v1/admin/ordering/resume	MANAGER +	Resume mobile ordering.
PUT	/api/v1/admin/wait-time	BARISTA+	Update wait time in minutes.

GET	/api/v1/admin/orders/:id/events	MANAGER +	Full audit trail.
GET	/api/v1/admin/dashboard	OWNER	Revenue, orders, top items.
GET	/api/v1/admin/feature-flags	OWNER	All feature flags + state.
PUT	/api/v1/admin/feature-flags/:key	OWNER	Toggle feature on/off.

4.4 Role-Based Access Control

Permission	BARISTA	MANAGER	OWNER
View live orders	YES	YES	YES
Accept / mark ready / in progress	YES	YES	YES
Toggle sold-out / available	YES	YES	YES
Set wait time	YES	YES	YES
Cancel order	—	YES	YES
Issue refund (full or partial)	—	YES	YES
Pause / resume mobile ordering	—	YES	YES
Edit menu prices	—	—	YES
Manage staff accounts	—	—	YES
View revenue reports	—	—	YES
Toggle feature flags	—	—	YES
Manage locations	—	—	YES

4.5 Rate Limiting

Endpoint	Limit	Reason
POST /auth/login	5 / min per IP	Brute force protection
POST /auth/register	10 / min per IP	Spam prevention
POST /checkout	3 / min per user	Prevent checkout spam
POST /stripe/webhook	100 / min	Stripe sends bursts
GET /menu	60 / min per IP	Served from Redis — generous
Admin endpoints	30 / min per staff	General protection

5.1 Three State Machines

OrderStatus

Status	Meaning
DRAFT	Cart submitted. No payment started yet.
PENDING_PAYMENT	Stripe PaymentIntent created. Awaiting customer payment.
PAID	Stripe webhook confirmed. SET ONLY BY STRIPE WEBHOOK — never by iOS.
ACCEPTED	Staff accepted order in dashboard.
IN_PROGRESS	Barista started making the order.
READY	Barista marked done. Push notification sent to customer.
PICKED_UP	Customer collected. Order complete.
CANCELLED	By customer (DRAFT only) or staff.
REFUNDED	Full refund issued via Stripe.
FAILED	Payment failed or unrecoverable error.

PaymentStatus (independent of OrderStatus)

Status	Meaning
REQUIRES_PAYMENT	Initial state. PaymentIntent created.
PROCESSING	Payment in flight.
SUCCEEDED	Stripe confirmed. ONLY valid signal to mark order PAID.
FAILED	Payment declined or error.
REFUNDED	Full refund issued.
PARTIALLY_REFUNDED	Partial refund. See refunds table.

CloverSyncStatus (independent of OrderStatus and PaymentStatus)

Status	Meaning
NOT_SENT	Not yet pushed to Clover.
PENDING	In the sync queue.
SENT	Successfully received by Clover POS.
FAILED	At least one attempt failed. Retrying.
MANUAL_REQUIRED	All 4 retries exhausted. Telegram alert sent. Staff must enter manually.

CRITICAL: CloverSyncStatus = MANUAL_REQUIRED does NOT mean the order failed. The customer HAS paid. OrderStatus = PAID. PaymentStatus = SUCCEEDED. Only the POS sync failed. Staff notified via Telegram.

5.2 Checkout & Stripe Flow (Step by Step)

1. Customer taps Checkout
2. iOS generates idempotency key: `SHA256(userId + cartHash + timestamp)`
3. `POST /api/v1/checkout { items, tipPercent, pickupType, idempotencyKey }`
4. Backend: check `idempotency_key` in `orders` table → duplicate? return cached response
5. Backend: validate location is open (check `location_hours` + `location_settings`)
6. Backend: validate all items exist, active, available in inventory
7. Backend: calculate ALL pricing in integer cents (`subtotal`, `modifiers`, `tax`, `tip`, `total`)
`tax = Math.round(subtotalCents * taxRateBps / 10000)`
`tip = Math.round(subtotalCents * tipPercent / 100)`
8. DB transaction:
`INSERT orders status=PENDING_PAYMENT idempotency_key=...`
`CREATE Stripe PaymentIntent amount=total_cents metadata={orderId}`
`UPDATE orders SET stripe_payment_id = paymentIntent.id`
`RETURN { orderId, clientSecret } to iOS`
9. iOS presents Stripe payment sheet (Apple Pay / card)
10. Customer pays
11. Stripe calls `POST /api/v1/stripe/webhook event=payment_intent.succeeded`
12. Backend: verify Stripe-Signature header – reject if invalid
13. ATOMIC OUTBOX TRANSACTION:
`UPDATE orders SET order_status=PAID, payment_status=SUCCEEDED`
`INSERT order_events from=PENDING_PAYMENT to=PAID by=system`
`INSERT outbox_events type=ORDER_PAID status=PENDING`
`COMMIT` – both changes guaranteed atomically
14. Outbox worker picks up `ORDER_PAID` within 1 second
15. Dispatches to: `clover.worker` + `order.worker` (push + loyalty + analytics) + Telegram

iOS app NEVER marks an order paid. Stripe webhook is the ONLY source of truth.

5.3 Outbox Pattern — Why It Exists

Without outbox (dangerous):

```
UPDATE orders SET status = 'PAID'    ← DB succeeds
await publishToSQS('ORDER_PAID')    ← network blip! Lost forever.
Result: order paid, Clover never notified, customer never pushed, owner never alerted
```

With outbox pattern (safe):

```
BEGIN TRANSACTION
UPDATE orders SET order_status = 'PAID'
INSERT INTO outbox_events (event_type, payload, status)
VALUES ('ORDER_PAID', {orderId, customerId, totalCents}, 'PENDING')
COMMIT ← both happen atomically or both fail. Nothing in between.
```

Outbox worker (runs every 1 second):

```
SELECT * FROM outbox_events WHERE status = 'PENDING' LIMIT 10
FOR each event → dispatch → mark PROCESSED
IF fails 5 times → mark DEAD → Telegram alert to owner
```

5.4 Clover Sync & Dead Letter Queue

`syncToClover(orderId):`

```
delays = [0ms, 30,000ms, 120,000ms, 600,000ms] ← 0s, 30s, 2min, 10min
```

FOR each delay:

```
await sleep(delay)
```

TRY:

```
POST to https://api.clover.com/v3/merchants/{merchantId}/orders
Headers: { Authorization: Bearer {apiKey from Parameter Store} }
UPDATE orders SET clover_sync_status = 'SENT'
INSERT clover_sync_log attempt=N status=SENT
```

```
RETURN success
```

```
CATCH error:
```

```
INSERT clover_sync_log attempt=N status=FAILED error=message
```

```
IF last attempt:
```

```
UPDATE orders SET clover_sync_status = 'MANUAL_REQUIRED'
```

```
ORDER STATUS AND PAYMENT STATUS ARE NOT CHANGED
```

```
SEND Telegram:
```

```
CLOVER SYNC FAILED – MANUAL ACTION REQUIRED
```

```
Order: {id} | Customer paid: YES | In Clover POS: NO
```

```
Enter manually in Clover terminal.
```

5.5 Pickup Time & Location Hours Logic

```
canAcceptOrders(locationId, pickupType, scheduledTime?):
```

```
1. Check location_settings.mobile_ordering_paused
```

```
→ TRUE → reject: 'Mobile ordering is paused right now'
```

```
2. Get today's location_hours by day_of_week
```

```
→ is_closed = TRUE + ASAP → reject: 'We open at {nextOpenTime}'
```

```
→ is_closed = TRUE + SCHEDULED → continue (ordering for tomorrow is OK)
```

```
3. Check if current time is within open_time..close_time
```

```
→ Outside hours + ASAP → reject with next open time
```

```
4. If SCHEDULED:
```

```
→ location_settings.scheduled_ordering must be TRUE
```

```
→ scheduledTime must be within open hours on that day
```

```
→ scheduledTime must be <= max_schedule_days from now
```

```
5. All checks pass → { allowed: true, estimatedReady: now + current_wait_minutes }
```

6.1 SwiftUI Project Structure

```

PulsCoffeeApp/
  ■■■ Core/
  ■ ■■■ AppState.swift           @MainActor ObservableObject – auth, location, feature flags
  ■ ■■■ APIClient.swift         all network calls – single entry point
  ■ ■■■ Storage.swift           UserDefaults + local menu disk cache
  ■ ■■■ EventBus.swift          local NotificationCenter wrapper
  ■■■ Features/
  ■ ■■■ Auth/
  ■ ■ ■■■ LoginView.swift
  ■ ■ ■■■ RegisterView.swift
  ■ ■ ■■■ AuthViewModel.swift
  ■ ■■■ Menu/
  ■ ■ ■■■ MenuView.swift        ALWAYS loads cache first, refreshes background
  ■ ■ ■■■ MenuCategoryView.swift
  ■ ■ ■■■ ItemDetailView.swift  modifiers + quantity + add to cart
  ■ ■ ■■■ MenuViewModel.swift
  ■ ■■■ Cart/
  ■ ■ ■■■ CartView.swift
  ■ ■ ■■■ CartManager.swift     @StateObject – lives in MEMORY ONLY, no API calls
  ■ ■■■ Checkout/
  ■ ■ ■■■ CheckoutView.swift    button locks on first tap, never double-submits
  ■ ■ ■■■ CheckoutViewModel.swift
  ■ ■ ■■■ IdempotencyKey.swift   SHA256(userId + cartHash + timestamp)
  ■ ■■■ Orders/
  ■ ■ ■■■ OrderStatusView.swift  polls GET /orders/:id every 10s while active
  ■ ■ ■■■ OrderHistoryView.swift
  ■ ■ ■■■ OrderViewModel.swift
  ■ ■■■ Loyalty/
  ■ ■ ■■■ LoyaltyView.swift
  ■ ■■■ Profile/
  ■ ■ ■■■ ProfileView.swift
  ■■■ Models/
  ■ ■■■ MenuItem.swift          matches backend exactly
  ■ ■■■ Order.swift             OrderStatus enum matches backend exactly
  ■ ■■■ Customer.swift
  ■ ■■■ PriceCalculation.swift   display only – NEVER use for charging
  ■■■ Services/
  ■ ■■■ MenuCacheService.swift   disk cache – show immediately, refresh silently
  ■ ■■■ PushNotificationService.swift
  ■ ■■■ AnalyticsService.swift   PostHog events

```

6.2 Key iOS Rules

Rule	Implementation
Menu always instant	LocalCache getMenu() shown immediately. API refresh in background Task{}
Cart in memory only	CartManager is @StateObject. No server calls to add/remove. POST on checkout only.
Checkout button locks	guard !isProcessing else { return }. isProcessing=true on tap. Only reset on error.
Idempotency key	SHA256(userId + sorted(cartItemIds) + timestamp). In every checkout request.
Never calculate prices	iOS only displays PriceCalculation returned by backend. No local math.

Order status polling	Poll every 10s while status not in [PICKED_UP, CANCELLED, FAILED].
JWT in Keychain	kSecAttrAccessibleWhenUnlocked. Never UserDefaults. Never UserDefaults.
Sentry on first line	init in @main App.init() before anything else runs.
Feature flags cached	Fetch on launch. Cache 5 minutes. Gates every feature behind flag check.

6.3 Performance, Caching & Offline

What	Offline Behaviour	Network Target
Browse menu	Shows last cached menu	< 300ms (cache hit)
View past orders	Last 10 cached locally	< 500ms
Build cart	Works — local state	Instant
Place order	Fails — clear error message	< 1.5s
Order status	Shows last known + timestamp	< 3s end-to-end
Menu images	Kingfisher cache — shows stale	< 50ms (CDN cache hit)

7.1 Phase 1 — Core Ordering (Weeks 1-8)

Goal: a real customer successfully orders and picks up a real coffee.

#	Feature	Why Phase 1
1	Menu browsing (categories, images, descriptions)	Core product
2	Drink customisation (size, milk, extras, notes)	Core product
3	Cart management (in-memory, no server)	Core product
4	Checkout with Stripe + Apple Pay	Core revenue
5	Idempotency — zero duplicate orders or charges	Payment safety
6	Order tracking (real-time status polling)	Customer trust
7	Push notifications (confirmed, ready, cancelled)	Operations
8	User accounts (register, login, profile)	Identity
9	Loyalty points — earn on every order	Retention foundation
10	Loyalty tiers (Bronze → Silver → Gold → Platinum)	Gamification
11	Inventory control — staff toggle sold-out instantly	Operations
12	Pause mobile ordering — one toggle in staff dashboard	Rush hour control
13	Wait time display on menu + checkout screens	Expectation setting
14	Scheduled pickup — order up to 7 days ahead	Convenience
15	Multi-location — each store has own menu and hours	Scalability

7.2 Phase 2 — Retention & Intelligence (Months 2-3)

Build ONLY after Phase 1 is live and data is flowing.

#	Feature	Business Value
16	'Your usual' — one-tap reorder	Massive ROI. Starbucks' most powerful feature.
17	Coffee streak system — don't break it	Best retention psychology in the spec.
18	Real-time busy status (Quiet/Moderate/Busy)	Drives daily app opens like traffic apps.

19	Weather-aware suggestions	Simple. Effective. Hot day = iced drinks.
20	Churn prevention — auto-detect going-quiet customers	Especially for subscription retention.
21	Smart push notifications — sent at personal best time	Higher open rates than broadcast.
22	Bundle intelligence — AI pairs items per customer	High ROI. Matcha + muffin upsell.
23	Subscription pass — \$29/mo, 1 free drink/day	Potentially HUGE. Guaranteed revenue.
24	Flash deal engine — triggers during slow periods	Fill dead time without annoying regulars.
25	Surprise rewards — random unexpected points	Unpredictable rewards = more addictive.

7.3 Phase 3 — Delight & Premium UX (Months 4-6)

#	Feature	Notes
26	Live Activity / Dynamic Island — order progress	Actually great. Natural for coffee tracking.
27	Home screen widget — suggestion + busy status	Daily utility without opening app.
28	Predictive pre-ordering — geofence trigger	5 mins away → 'Start your usual?'
29	Group ordering + office rounds	Good office potential. Track who bought last.
30	Coffee gifting — free drink via iMessage link	Social viral mechanics.
31	Birthday week — free drink + double points	Emotional connection. Always shared.
32	Milestone celebrations — 100th coffee free	Creates memorable moments.
33	Social proof — '47 people ordered this today'	Reduces decision fatigue.
34	Limited time / seasonal items with countdown	Scarcity drives urgency to open app.
35	Referral system — give \$5, get \$5	Growth mechanic.

7.4 Phase 4 — Scale & SaaS (Month 6+)

This is the most important long-term direction: multi-shop licensing. Your real opportunity may be a coffee operations platform, not just one app.

#	Feature	Why It Matters
---	---------	----------------

36	Waste reduction AI — predict daily prep quantities	Real AI value. Saves money every day.
37	Menu optimisation AI — margin vs popularity analysis	"Vanilla Latte: 40% margin, 5% orders — promote it"
38	Owner analytics dashboard — revenue, trends, CLV	Business intelligence on your phone.
39	Multi-location reporting	Consolidated view across all stores.
40	Multi-shop licensing — white-label platform	Your app becomes a SaaS product. Other shops pay monthly.

7.5 Features Removed or Delayed (with reasons)

Feature	Decision	Reason
Mood detection	REMOVED permanently	'Rough Monday?' feels creepy. Invasive.
Bluetooth beacons	REMOVED for years	Operational nightmare. Hardware dependency.
Apple Watch app	DELAYED to Phase 3+	Tiny ROI. Low priority across all reviewers.
Conversational AI	DELAYED to Phase 3	Most people know what they want. Low ROI early.
Individualized pricing	REMOVED permanently	Customers hate feeling manipulated. Only discounts off base price.
AI offer engine (day one)	DELAYED to Phase 2	Start rule-based. Simple beats smart.
AR menu	DELAYED to Phase 4+	Not a coffee shop priority.
Siri Shortcuts	DELAYED to Phase 3	Nice but low impact.
GraphQL/AppSync	REMOVED	REST is simpler. No MVP benefit.
EventBridge for all events	REDUCED	Only 6 critical events. PostHog handles the rest.

8.1 When to Build AI — The Gate Condition

DO NOT start Phase 2 AI until ALL of these are true:

Gate	Why
Phase 1 is live and stable	AI on top of a broken ordering system makes nothing better
At least 200 real orders processed	Personalize needs data to train on
At least 50 active weekly customers	Churn detection needs a baseline
order_events table is being populated	AI memory depends on this
behavior_events logging is running	Recommendation engine needs signal

8.2 AI Architecture & Memory System

How AI memory works:
AI models (Claude) are stateless – they start fresh every single call.
Your PostgreSQL database IS the memory.
Before every Claude API call, Lambda builds a context packet from the DB and injects it.

```
// Lambda: buildContextPacket(customerId)
const context = {
  customer: {
    name: "Sarah", preferredDrink: "Large Oat Latte, extra shot",
    avoids: ["matcha","hazelnut","dairy"], priceSegment: "premium",
    adventurousness: 0.2
  },
  patterns: {
    usualOrderTime: "08:10", usualDays: ["Monday","Tuesday","Thursday"],
    lastVisit: "yesterday", visitStreak: 7, churnRisk: 0.12
  },
  now: {
    time: "07:52 Tuesday", weather: { temp: 28, condition: "snow" },
    shopStatus: "quiet", waitMinutes: 2, todaysSpecial: "Brown Sugar Latte"
  },
  marketingHistory: {
    lastOffer: "double_points (14 days ago)",
    offerResponse: "always ignores discounts", notificationOpenRate: 0.84
  }
}

// Prompt sent to Claude API (claude-sonnet-4-20250514):
// "You are the coffee AI for Puls Coffee. Customer data: {context}
// Decide: homeScreen, notification (if any), offer (if any), bundle.
// Sarah is premium – never discount her. Respond in JSON only."

// Claude returns structured JSON → app renders personalised home screen
```

AI Services Used in Phase 2

Service	Purpose	Cost
Claude API (claude-sonnet-4-20250514)	Personalised home screen, offer copy, conversational ordering	~\$0.001/decision

Amazon Personalize	ML recommendation engine. Trains nightly on all order history.	~\$0.05/user/mo
AWS Forecast	Predicts busy periods. Staff prep alerts. Customer nudges.	~\$10-20/mo
OpenWeatherMap API	Temperature + conditions → adjust drink suggestions	Free (1M/mo)
AWS Pinpoint	Smart push notifications. A/B tests message variants.	Free (1M/mo)

8.3 Customer Segments & Marketing Engine

Segment	Detection	AI Action
AT RISK	Haven't visited 7+ days. Was 3x/week.	'We miss you. Free drink on us today ■'
COOLING	Visit frequency dropped 40%+ this month	'Double points this week — just for you'
LOYAL	Consistent regular visitor	Never discount. Early access to new items. VIP messaging.
WHALE	Top 5% spender. Very loyal.	Personal thank you. Exclusive items. Priority treatment.
NEW	First or second visit	'Welcome! Free upgrade on your next 3 visits'
CHURNED	Gone 30+ days	Last resort: '\$5 credit. We'd love to see you again.' Then stop.

Anti-Spam Rules — Hard-Coded, Never Override

Rule	Value
Max notifications per customer per day	1
No notifications after	8:00 PM local time
Stop after consecutive ignores	3 ignored → pause 30 days
Never send same offer twice	Check offers table before generating
Never discount loyal/premium customers	Check price_segment before offer type
Monthly offer budget	Default \$200 — AI stays within this

8.4 Pricing Psychology (legally safe — only discounts, no surcharges)

Strategy	How	Example
Off-peak flash deals	\$1 off during slow hours — push notification	'2-4pm Tuesday: \$1 off any drink next 30 mins'
Anchor pricing	Show crossed-out original price	~~\$8.50~~ → \$5.50 TODAY ONLY
Decoy pricing	S=\$3.50 / M=\$5.00 / L=\$5.50 — M makes L look cheap	Most customers pick Large
Bundle intelligence	AI pairs items this specific customer likely wants	Sarah: Latte + Almond Croissant = \$8 (save \$2)
Scarcity	'Only 3 Matcha Lattes left' — inventory-driven	Drives urgency without lying

Payday awareness	1st + 15th of month = reduce discounts, push premium	People just got paid
------------------	--	----------------------

8.5 Smart Engagement Mechanics

Mechanic	Why It Works
Coffee streak — '7-day streak. Don't break it!'	Loss aversion. Fear of losing streak > effort to order.
Surprise rewards — random unexpected points	Unpredictable rewards more addictive than predictable ones.
Milestone celebrations — '100th coffee! Free drink'	Creates emotional moments. People share these.
VIP early access to new items	Loyalty feels exclusive, not transactional.
Subscription pass — \$29/mo	Guaranteed revenue. People come MORE because 'already paid'.
Birthday week — free drink + double points	Emotional connection. Always shared on social media.
Limited time seasonal items	Scarcity + seasonality = urgency to open app.

Staff Web Dashboard (React + TypeScript, Week 7)

Screen	Access	Key Features
Live Orders	All staff	Real-time queue. 5s auto-refresh. Accept / Mark Ready / Cancel buttons.
Order Detail	All staff	Full breakdown: items, modifiers, notes, complete order_events audit trail.
Inventory Control	All staff	Toggle sold-out instantly. Set quantity. Effect in customer app immediately.
Shop Controls	Manager+	Pause/resume ordering. Set wait time. Both affect customer app instantly.
Refunds	Manager+	Full or partial. Stripe called server-side. Reason required. Logged.
Reports	Owner only	Daily/weekly revenue, top items, new vs returning customers.
Feature Flags	Owner only	Toggle any feature on/off — no deployment needed.

Telegram Owner Bot (Week 5 — before iOS app)

Alert	Message
New order	NEW ORDER — Sarah M. — Oat Latte + Muffin — \$10.00 — Main St
Clover sync failed	CLOVER SYNC FAILED — Order #123 — PAID YES — NOT IN POS — Enter manually
Payment failed	PAYMENT FAILED — Order #124 — \$8.50 — Customer: Mike K.
Item sold out	OAT MILK SOLD OUT — Auto-hidden from app — Main St
Ordering paused	MOBILE ORDERING PAUSED — Main St — by: Manager Jane
Dead outbox event	DEAD EVENT — ORDER_PAID for order #125 failed 5 times — manual action needed
Daily summary (9pm)	TODAY: 47 orders \$423.50 revenue Avg: \$9.01 Top: Oat Latte x12 New: 8
Weekly (Sunday)	WEEK: 312 orders \$2,847 revenue vs last week: +12%

AWS Infrastructure

Service	Config	Purpose
ECS Fargate	0.25 vCPU/512MB → scale up	NestJS API + Worker containers. Zero server management.
RDS PostgreSQL 15	db.t3.micro → t3.small on growth	Managed DB. Daily backups. Point-in-time recovery.
ElastiCache Redis	cache.t3.micro single node	Menu cache (10m TTL). Idempotency keys (24h TTL).
SQS Standard Queue	VisibilityTimeout=30s MaxReceive=4	Reliable async. DLQ after 4 failed attempts.
S3 + CloudFront	S3 storage + global CDN	Menu images. cache-control: max-age=31536000.
Cognito	Separate pools: customers + staff	JWT issuer. MFA + social login ready.
Parameter Store	SecureString — AES-256	Clover key, Stripe key, Telegram token, DB URL.
Amplify Hosting	Free tier	Staff React dashboard. Auto-deploy from GitHub.

Security Rules

Area	Rule
Secrets	AWS Parameter Store (SecureString). Never in .env, never in code, never in logs.
Stripe webhooks	Verify Stripe-Signature header every time. Reject without valid signature.
JWT	Access: 15min. Refresh: 30 days. Auto-rotate on use.
iOS Keychain	JWT in kSecAttrAccessibleWhenUnlocked. Never UserDefaults.
RBAC	Role guard at controller level. Cannot be bypassed by request manipulation.
SQL injection	TypeORM parameterised queries only. Never string-concatenate SQL.
HTTPS	TLS 1.2+ everywhere. HTTP rejected at load balancer.
Input validation	class-validator DTOs. whitelist: true. forbidNonWhitelisted: true.
Price validation	Backend recalculates all prices. iOS-sent prices are ignored.

Observability & Monitoring

Tool	What to Track
Sentry (iOS)	All unhandled exceptions. Checkout failures. Payment errors.
Sentry (Backend)	All 5xx errors. Stripe webhook failures. Slow DB queries (>500ms).
PostHog (iOS)	app_opened, menu_viewed, item_added_to_cart, checkout_started, order_placed.
PostHog (Backend)	order_paid, order_ready, order_picked_up, refund_issued, clover_sync_failed.
CloudWatch	ECS CPU/memory, RDS connections, SQS queue depth, Redis memory.
CloudWatch Alarms	DLQ receives message → SNS → Telegram. RDS CPU > 80% → alert.

order_events table	First line of support. 'Where is my order?' answered in 3 seconds.
clover_sync_log	Full request/response logged. Debug any POS issue instantly.

PART 11 — 8-WEEK BUILD PLAN

Week 1	Foundation	PostgreSQL schema (all tables). NestJS project scaffold. AWS Cognito auth. JWT + refresh strategy. RBAC guards (OWNER/MANAGER/BARISTA). Sentry init (first line). PostHog init. Docker + docker-compose. GitHub Actions CI (typecheck + tests).
Week 2	Menu + Pricing + Inventory	Menu API with Redis cache (10min TTL). Pricing module — all math in integer cents, tax in basis points. Inventory module. Location + hours service (open/closed validation). Feature flags service. Clover menu import script. S3 + CloudFront for images.
Week 3	Checkout + Stripe + Outbox	POST /checkout with idempotency guard. Stripe PaymentIntent creation. Stripe webhook handler with signature verification. Outbox pattern — atomic DB update + event insert. Order state machine with strict enum transitions. order_events logging on every status change.
Week 4	Clover Sync + Workers + DLQ	Clover order creation API call (keys from Parameter Store). outbox.worker (polls 1s). order.worker (loyalty + push + analytics). clover.worker (retry: 0s→30s→2m→10m). Dead letter queue handler. Telegram alert service. clover_sync_log population.
Week 5	Push + Loyalty + Telegram	APNs push notifications (ORDER_READY, ORDER_CANCELLED). Loyalty points on ORDER_PAID. Tier upgrades (Bronze→Silver→Gold→Platinum). Loyalty transaction log. Telegram daily summary + weekly summary. Telegram bot commands (/orders, /revenue, /pause).
Week 6	iOS App — All Screens	SwiftUI project. Auth (login, register, forgot password). Menu with disk cache. Item detail + modifiers. Cart (in-memory CartManager). Checkout + Stripe payment sheet + Apple Pay. Order status polling. Order history. Profile + loyalty view. Sentry + PostHog SDK.
Week 7	Staff Web Dashboard	React + TypeScript project. Live order queue (5s auto-refresh). Accept/in-progress/ready/cancel. Order detail + audit trail. Inventory sold-out toggle. Pause/resume control. Wait time editor. Manager refund screen. Owner reports. Feature flag management. Hosted on Amplify.
Week 8	Testing + App Store	End-to-end test: order → Clover sync → READY push → PICKED_UP. Rate limiting (429 validation). Load test checkout with k6 or Artillery. App Store assets (6 screenshots, description, privacy policy). TestFlight beta with 10-20 real Puls Coffee customers. Bug fixes. App Store submission.

Phase 2+ Roadmap After Launch

Phase	Timeline	Focus	Key Deliverables
Phase 2	Month 2-3	Retention + AI	Your usual, streaks, busy status, churn prevention, weather AI, subscription pass
Phase 3	Month 4-6	Premium UX	Live Activity, home widget, predictive ordering, group ordering, gifting
Phase 4	Month 6+	Scale + SaaS	Waste AI, menu optimisation AI, multi-shop licensing

PART 12 — COST ESTIMATES

Service	Launch (0-50/day)	Growing (50-500/day)	Scaling (500-5k/day)
RDS PostgreSQL (t3.micro)	\$15/mo	\$25/mo	\$60/mo
ECS Fargate (API + Workers)	\$12/mo	\$30/mo	\$90/mo
ElastiCache Redis (t3.micro)	\$15/mo	\$15/mo	\$30/mo
SQS + DLQ	\$0/mo	\$1/mo	\$5/mo
S3 + CloudFront	\$1/mo	\$3/mo	\$10/mo
Cognito (free to 50k users)	\$0/mo	\$0/mo	\$5/mo
Parameter Store	\$0/mo	\$0/mo	\$1/mo
TOTAL AWS	~\$43/mo	~\$74/mo	~\$201/mo
Apple Developer Account	\$99/yr (\$8/mo)	\$99/yr	\$99/yr
Stripe fees	2.9% + \$0.30/txn	~\$200-2k/mo	~\$2k-20k/mo
Sentry	Free tier	Free tier	~\$26/mo
PostHog	Free (1M events/mo)	Free tier	~\$0-20/mo
Amazon Personalize (Phase 2)	—	~\$20-50/mo	~\$100/mo
Claude API (Phase 2)	—	~\$10-30/mo	~\$50-100/mo

Stripe fees dwarf all AWS costs at every stage. At 100 orders/day averaging \$9 each: Stripe takes ~\$60/day vs AWS at ~\$1.50/day. This is normal for any payment app.

PART 13 — 15 GOLDEN RULES — NON-NEGOTIABLE

#1	Menu loads instantly	Show local disk cache immediately on launch. Refresh API in background. Never block UI on network for menu display.
#2	Checkout is sacred	No AI. No experiments. No dynamic logic. Cart → tax → tip → pay → confirmed. Change as rarely as possible.
#3	Stripe webhook = payment truth	iOS NEVER marks an order paid. Only POST /stripe/webhook with valid Stripe-Signature sets payment_status = SUCCEEDED.
#4	Idempotency on every payment	Every checkout has a client-generated key. Server deduplicates silently. No exceptions ever.
#5	Order status is a strict enum	No ad-hoc strings. Invalid transitions rejected by state machine. TypeScript enum in code.
#6	Clover failure is NOT order failure	CloverSyncStatus = FAILED means POS sync failed, not payment. OrderStatus stays PAID. Alert owner. Never confuse these.
#7	All money in integer cents	650 not 6.50. Never float for money anywhere in business logic. Display formatting only in UI layer.
#8	iOS never calculates price	iOS sends items + tip percent. Backend calculates everything. iOS displays what backend returns.
#9	Outbox for critical events	ORDER_PAID event inserted in same DB transaction as status update. Never publish to queue directly.
#10	Sentry on day one	Initialize in the first commit. Before any other logic. You need it before you know you need it.
#11	Staff dashboard before AI	Build staff order queue, inventory controls, and wait time before any AI feature. Staff need it day one.
#12	Feature flags for everything risky	AI, pricing, subscriptions = flag defaulting FALSE. Deploy code, enable when ready. Zero hotfixes.
#13	Locations from day one	Every order, menu item, staff user, and setting scoped to location_id. Never hard-code single location.
#14	Three separate status enums	OrderStatus / PaymentStatus / CloverSyncStatus are independent. They fail independently. Never merge.
#15	Ship boring and reliable first	A simple app that takes orders reliably > a clever app that occasionally double-charges. Trust is everything.

Build the boring core perfectly.

Menu → Cart → Checkout → Payment → Clover → Status → Staff → Pickup.

When that works flawlessly, make it smart.

Puls Coffee — Complete Developer Specification | Version 3.0 | May 2025